INF6303

DEVOIR 1

Pipeline analytique polyglotte : Hive, MongoDB et Cassandra

ENONCE

Evaluation

Travail en équipe - valeur : 15 % de la note finale - date de remise : communiquée par le professeur

Ce devoir n'est pas une répétition des mini-labs. Les sorties produites dans Hive deviennent les entrées de MongoDB; les données enrichies dans MongoDB sont ensuite publiées dans Cassandra pour une consultation à faible latence.

Volume fourni

1 000 000 événements, 50 000 profils utilisateurs et 2 000 contenus. Les fichiers sont déjà générés et inclus : aucun script de génération ne vous ai remis.

1. Mandat et résultat attendu

Une plateforme de diffusion veut transformer un historique massif d'événements en une audience opérationnelle. Votre équipe doit construire un pipeline reproductible combinant trois technologies, chacune choisie pour son rôle.

Étape	Technologie	Résultat transmis
Analyse massive	Hive sur HDFS	segments_utilisateurs.csv
Enrichissement documentaire	MongoDB	audience_cible.jsonl
Publication faible latence	Cassandra	table requêtable par pays et segment

Question directrice

Comment articuler un stockage analytique distribué, un modèle documentaire flexible et une base orientée requête sans confondre leurs responsabilités?

Contraintes

- Utiliser exclusivement les données fournies.
- Conserver user_id comme clé d'interconnexion entre les trois systèmes.
- Ne pas charger le million d'événements dans MongoDB ou Cassandra : justifier les données transférées.
- Produire des preuves d'exécution et des contrôles de cohérence.
- Le travail doit pouvoir être rejoué à partir d'un environnement Docker remis à zéro.

2. Données distinctes des mini-labs

Ce devoir possède son propre jeu de données. Il ne réutilise pas le petit fichier de 24 événements des activités préparatoires.

Fichier	Volume logique	Rôle
evenements/*.csv.gz	1 000 000 lignes, 8 partitions	faits massifs analysés par Hive
profils_utilisateurs.jsonl.gz	50 000 documents	profils souples importés dans MongoDB
catalogue_contenus.csv	2 000 contenus	dimension jointe dans Hive

Relations

evenements.user_id rejoint profils.user_id; evenements.content_id rejoint catalogue.content_id. Le manifeste et les sommes SHA-256 permettent de contrôler l'intégrité des fichiers.

Interconnexion obligatoire

Une partie n'est considérée terminée que lorsque son fichier de sortie peut être consommé par la technologie suivante.

3. Démarrage et contrôle du volume

Décompresser l'archive, ouvrir PowerShell dans Version_Etudiants, puis démarrer le profil Hadoop.

```
docker compose --profile hadoop up -d --build
# JupyterLab : http://localhost:8888/lab?token=inf6303
```

Dans le terminal JupyterLab :

```
bash /home/jovyan/work/00_deposer_dans_hdfs.sh
hdfs dfs -du -h /inf6303/devoir1/evenements
hdfs dfs -ls /inf6303/devoir1/evenements
```

Contrôles à consigner

- Nombre de fichiers et volume HDFS.
- Pourquoi huit fichiers compressés permettent-ils davantage de parallélisme qu'un seul fichier gzip?
- Différence entre le volume compressé et le nombre logique de lignes.

4. Partie A - Hive/HiveQL (30 points)

Compléter workspace/01_hive_analyse.sql et l'exécuter avec Beeline.

```
beeline -u jdbc:hive2://hive:10000/default -n jovyan \
  -f /home/jovyan/work/01_hive_analyse.sql
```

A1. Tables et qualité - 8 points

- Créer deux tables externes typées sur les répertoires HDFS imposés.
- Confirmer exactement 1 000 000 événements et 2 000 contenus.
- Détecter les identifiants ou durées invalides; expliquer les contrôles.

A2. Analyse distribuée - 10 points

- Répartition des événements par pays et par type.
- Top 10 des contenus selon la durée totale après jointure avec le catalogue.
- Durée moyenne par appareil et qualité.
- Présenter le plan EXPLAIN d'une requête et repérer une opération coûteuse.

A3. Sortie transmise à MongoDB - 12 points

- Calculer par user_id : pays, nombre d'événements et durée totale.
- Attribuer le segment intensif, actif ou occasionnel selon les seuils imposés dans le fichier SQL.
- Écrire le résultat dans /inf6303/devoir1/sorties/segments.
- Rapatrier le résultat fusionné dans workspace/output/segments_utilisateurs.csv.

```
hdfs dfs -getmerge /inf6303/devoir1/sorties/segments \
  /home/jovyan/work/output/segments_utilisateurs.csv
```

5. Partie B - MongoDB (25 points)

Après la réussite de la partie A, démarrer le profil NoSQL. Si la mémoire est limitée, arrêter d'abord Hadoop sans supprimer les volumes.

```
docker compose --profile hadoop stop
docker compose --profile nosql up -d --build
python /home/jovyan/work/02_mongodb_enrichissement.py
```

B1. Import documentaire - 7 points

- Importer les 50 000 profils JSON/BSON.
- Préserver les sous-documents preferences et les tableaux devices.
- Créer un index unique sur user_id et justifier un second index composé.

B2. Enrichissement interconnecté - 10 points

- Lire segments_utilisateurs.csv produit par Hive.
- Mettre à jour chaque profil par user_id avec un sous-document analytics.
- Comparer les nombres de profils, segments et profils enrichis.
- Analyser explain() pour une requête utilisant l'index composé.

B3. Sortie transmise à Cassandra - 8 points

- Sélectionner les profils consentants des segments intensif et actif.
- Exporter user_id, pays, plan et analytics dans output/audience_cible.jsonl.
- Expliquer pourquoi cette sortie est plus petite que la source Hive.

6. Partie C - Cassandra (25 points)

Compléter workspace/03_cassandra_publication.py. La table doit répondre directement à la requête : top utilisateurs d'un pays et d'un segment, triés par score décroissant.

C1. Modèle orienté requête - 9 points

- Définir la clé de partition composée (country, segment).
- Choisir les colonnes de clustering permettant le tri demandé.
- Justifier la dénormalisation et discuter le risque de partition chaude.

C2. Chargement - 8 points

- Lire audience_cible.jsonl produit par MongoDB.
- Utiliser une requête préparée pour insérer les données.
- Comparer le nombre de documents exportés au nombre de lignes chargées.

C3. Requêtes de service - 8 points

- Afficher le top 10 Canada/intensif.
- Afficher un autre couple pays/segment pertinent.
- Montrer qu'une recherche globale par plan n'est pas naturelle pour cette table.
- Proposer la clé d'une table supplémentaire adaptée à ce nouveau besoin.

7. Partie D - Validation et architecture (15 points)

D1. Traçabilité - 7 points

- Choisir trois user_id et suivre leur présence de Hive à MongoDB puis Cassandra.
- Présenter un tableau avec valeurs comparées et expliquer toute différence.
- Exécuter workspace/04_validation_pipeline.py et joindre le résultat.

D2. Architecture - 8 points

- Dessiner les sources, HDFS, Hive, MongoDB, Cassandra et les fichiers de passage.
- Nommer chaque flèche : déposer, agréger, exporter, enrichir, publier, interroger.
- Distinguer stockage analytique, document opérationnel et vue de service.
- Identifier un point de reprise en cas d'échec entre deux systèmes.

Ce qui est évalué

La cohérence du pipeline compte davantage qu'une juxtaposition de commandes. Un résultat MongoDB non issu de la sortie Hive, ou une table Cassandra non issue de MongoDB, ne satisfait pas le mandat.

8. Livrables, barème et remise

Élément	Points
Partie A - Hive/HiveQL	30
Partie B - MongoDB	25
Partie C - Cassandra	25
Partie D - interconnexion et architecture	15
Reproductibilité et qualité de la remise	5
TOTAL	**100 (converti sur 15 %)**

Archive à remettre

- rapport.pdf : réponses, résultats, captures utiles et diagramme d'architecture;
- 01_hive_analyse.sql complété;
- 02_mongodb_enrichissement.py complété;
- 03_cassandra_publication.py complété;
- dossier output contenant segments_utilisateurs.csv et audience_cible.jsonl;
- README_EQUIPE.md indiquant les membres et la procédure de reproduction.

Ne pas remettre les volumes Docker ni recopier les fichiers sources massifs dans l'archive de remise. La date, le canal de dépôt et la convention de nommage seront communiqués par le professeur.

Critères de qualité

- Commandes reproductibles et commentées.
- Résultats vérifiés plutôt que simples captures.
- Justifications techniques précises.
- Aucun identifiant fabriqué manuellement entre les étapes.
- Répartition du travail indiquée dans le README d'équipe.

9. Aide au diagnostic

- docker compose ps : état des services.
- docker compose logs --tail=100 hive : journal HiveServer2.
- hdfs dfs -ls -R /inf6303/devoir1 : arborescence HDFS.
- Cassandra peut demander jusqu'à deux minutes avant d'accepter les connexions.
- docker compose --profile hadoop --profile nosql down -v : remise à zéro complète.

Rappel

Les mini-labs servent d'entraînement. Ce devoir demande une intégration réelle, un volume nettement supérieur, une justification des transferts et une validation de bout en bout.